

4.5 Algorithm of PrismSpace Multi-Agent System

The PrismSpace algorithm acts as the central brain that orchestrates multiple specialized AI agents into a coordinated swarm. Just like an efficient team, it understands user objectives, routes tasks to the best AI model, selects the right team of specialized agents, equips them with tools, and verifies that the final answer is safe and accurate. The workflow operates in three distinct phases:

- **Phase I: Offline Swarm Graph Representation** (Steps 1–5: Data prep, swarm graph, GAT routing)
- **Phase II: Single-Stage Preference Alignment** (Step 6: Safety and quality tuning via ORPO)
- **Phase III: Online Runtime Inference & Swarm Execution** (Steps 7–9: Live execution, streaming, evaluation)

Inputs (What Goes In):

- **User Objective** ($\mathcal{T}_{\text{user}}$): The user’s prompt, goal, and requirements written in everyday natural language.
- **Curated Training Corpora** ($\mathcal{D}$): Multi-source benchmark datasets containing prompt traces, intent categories, safety evaluations (WildGuard), and multi-agent trajectories (RouterBench, LMSYS Arena, CMU Agent Benchmark).
- **Preference Alignment Pairs** ($\mathcal{D}_{\text{pref}} = \{(x, y_w, y_l)\}$): Examples of prompt x paired with a winning/preferred response y_w and a losing/disfavored response y_l .
- **Global Agent Swarm Registry** ($\mathcal{G}_{\text{global}} = (V, E)$): The available roster of AI agents (Supervisor, Coding, Research, Validator, Planner) and Model Context Protocol (MCP) tool endpoints.

Outputs (What Comes Out):

- **Task Intent & Best Provider**: Predicted task category ($\hat{y}_{\text{intent}}$) and optimal LLM provider ($\hat{y}_{\text{provider}}$, e.g., Groq for rapid response, OpenAI/Anthropic for complex reasoning).
- **Swarm Dispatch Vector** ($\hat{y}_{\text{agents}}$): A binary checklist indicating which agents are activated (1) or kept idle (0).
- **Execution Plan** ($\mathcal{G}_{\text{active}}$): A Directed Acyclic Graph outlining task steps with predicted latency ($\hat{\ell}$) and cost ($\hat{c}$).
- **Tuned Model Adapters** (ΔW): Lightweight LoRA weights ensuring helpful and safe model responses.
- **Evaluation Metrics**: Validation benchmarks including Accuracy, F1-Score, ROC-AUC, and ECE calibration.

Step 1: Data Acquisition and Preprocessing (*Preparing clean data for training*)

1. **Data Ingestion**: Collect and unify records from multi-source training datasets ($\mathcal{D}$), standardizing prompt texts, intent labels, and tool execution logs into a consistent format.
2. **Data Cleaning**: Remove malformed text, sanitize special characters, strip redundant whitespace, and filter duplicates.
3. **Categorical Encoding**: Convert discrete categories (such as agent roles and tool capabilities) into numerical vectors.
4. **Feature Standardization (z -score scaling)**: Scale all numerical features to zero mean and unit variance:

$$z_i = \frac{x_i - \mu_x}{\sigma_x}$$

Intuition: Subtract the average (μ_x) and divide by standard deviation (σ_x) so that different metrics (like prompt length and execution time) share a common scale.

5. **Dataset Splitting**: Partition data into 80% training ($\mathcal{D}_{\text{train}}$), 10% validation ($\mathcal{D}_{\text{val}}$), and 10% testing ($\mathcal{D}_{\text{test}}$).

Step 2: Swarm Graph Construction (*Modeling connections between agents and tools*)

1. **Define Nodes (V):** Treat each agent persona and MCP tool v_i as an individual node in the graph set V .
2. **Compute Pairwise Similarity via Pearson Correlation:** Calculate functional alignment between feature vectors $\mathbf{x}_i$ and $\mathbf{x}_j$:

$$s_{ij} = \frac{\text{Cov}(\mathbf{x}_i, \mathbf{x}_j)}{\sigma_{\mathbf{x}_i} \sigma_{\mathbf{x}_j}}$$

Intuition: Pearson correlation measures how closely two agents or tools collaborate. Scores range from -1 (opposite capabilities) to $+1$ (perfect functional alignment).

3. **Form Adjacency Matrix (A):** Connect nodes if their capability correlation exceeds threshold τ :

$$A_{ij} = \begin{cases} 1, & \text{if } s_{ij} \geq \tau \\ 0, & \text{otherwise} \end{cases}$$

4. **Assemble Swarm Graph ($\mathcal{G} = (V, E)$):** Build the network graph $\mathcal{G}$ where edges E represent active collaboration channels between complementary agents and tools.

Step 3: Graph Attention Network (GAT) Initialization (*Setting up the neural graph model*)

1. **Initialize Node Embeddings:** Assign initial representation vector $\mathbf{h}_i = \mathbf{x}_i \in \mathbb{R}^d$ to every node $v_i \in V$.
2. **Initialize Trainable Weights:** Set up transformation matrix $\mathbf{W} \in \mathbb{R}^{d' \times d}$ and attention vector $\mathbf{a} \in \mathbb{R}^{2d'}$ using standard Xavier initialization.
3. **Configure Hyperparameters:** Set learning rate η , number of attention heads K , and dropout rate p .

Step 4: Attention-Based Message Passing (*Sharing relevant knowledge across the swarm*)

1. **Calculate Attention Relevance Scores:** For node v_i and each connected neighbor $v_j \in \mathcal{N}(i)$, compute attention coefficient e_{ij} :

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j])$$

Intuition: This formula calculates how much attention node i should pay to neighbor j . The symbol $\parallel$ concatenates their feature vectors side by side.

2. **Normalize Scores with Softmax:** Turn raw attention scores into relative percentages that sum to 1.0 (100%):

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}$$

Intuition: Softmax ensures neighbor weights α_{ij} are always positive and proportionally distributed.

3. **Aggregate Neighbor Information:** Update each node's state using a weighted sum of its neighbors:

$$\mathbf{h}'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j \right)$$

Intuition: Each agent updates its own knowledge ($\mathbf{h}'_i$) by gathering advice from neighboring agents and tools, weighted by how relevant each neighbor is.

4. **Multi-Head Attention:** Run K independent attention heads in parallel to capture distinct relationships (e.g., coding assistance vs. safety validation), and concatenate outputs: $\mathbf{h}'_i = \parallel_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^k \mathbf{W}^k \mathbf{h}_j \right)$.

Step 5: Multi-Task Classification and Loss Optimization (Training the routing decision heads)

1. **Prediction Heads:** Feed updated embeddings $\mathbf{h}'_i$ into two dedicated output layers:

- *Intent & Model Routing Head:* Softmax predicts the probabilities for each intent class and LLM provider:

$$\hat{\mathbf{y}}_{\text{route}} = \text{Softmax}(\mathbf{W}_{\text{route}}\mathbf{h}' + \mathbf{b}_{\text{route}})$$

- *Agent Dispatch Head:* Sigmoid independently predicts whether each candidate agent should be activated:

$$\hat{y}_{\text{agent},i} = \text{Sigmoid}(\mathbf{W}_{\text{agent}}\mathbf{h}'_i + b_i)$$

2. **Multi-Task Loss Computation:** Calculate training loss across routing and agent dispatch:

$$\mathcal{L}_{\text{class}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) - \frac{1}{|V|} \sum_{v \in V} [y_v \log(\hat{y}_v) + (1 - y_v) \log(1 - \hat{y}_v)]$$

Intuition: The first term trains the router to pick the correct model/intent. The second term trains the dispatcher to activate only the agents needed for the task.

3. **Parameter Optimization:** Update parameters θ using the Adam optimizer to minimize the total loss.

Step 6: Preference Alignment via Odds Ratio Preference Optimization (ORPO) (Teaching AI good vs. bad)

1. **Why ORPO?** Traditional alignment methods (like RLHF or DPO) require separate reward models or multiple large networks. ORPO trains the model in a single step by combining fluency training with a preference penalty.

2. **Lightweight Fine-Tuning (LoRA):** Attach Low-Rank Adaptation matrices $\Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{BA}$ to model layers, training $< 1\%$ of weights while keeping the main foundation model frozen and fast.

3. **Supervised Learning on Winning Answers ($\mathcal{L}_{\text{SFT}}$):** Train the model to generate high-quality winning responses:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{|y_w|} \sum_{t=1}^{|y_w|} \log P_{\theta}(y_{w,t} \mid x, y_{w,<t})$$

Intuition: Standard next-token training ensuring the AI speaks fluently and accurately on chosen examples y_w .

4. **Generative Odds Formulation:** For sequence y , calculate its length-normalized probability and odds:

$$\text{odds}_{\theta}(y \mid x) = \frac{P_{\theta}(y \mid x)}{1 - P_{\theta}(y \mid x)} \quad \text{where} \quad P_{\theta}(y \mid x) = \left(\prod_{t=1}^{|y|} P_{\theta}(y_t \mid x, y_{<t}) \right)^{\frac{1}{|y|}}$$

Intuition: The odds tell us how likely the AI is to generate sequence y compared to not generating it.

5. **Odds Ratio Penalty ($\mathcal{L}_{\text{OR}}$):** Directly penalize the model whenever it favors a rejected completion:

$$\mathcal{L}_{\text{OR}}(\theta) = -\log \sigma \left(\log \frac{\text{odds}_{\theta}(y_w \mid x)}{\text{odds}_{\theta}(y_l \mid x)} \right)$$

Intuition: Increases the odds of winning answer y_w while suppressing the odds of rejected answer y_l .

6. **Unified Single-Stage Objective:** Combine language fluency with preference penalty into one objective:

$$\mathcal{L}_{\text{ORPO}}(\theta) = \mathcal{L}_{\text{SFT}}(\theta) + \beta \cdot \mathcal{L}_{\text{OR}}(\theta)$$

where hyperparameter β balances language fluency ($\mathcal{L}_{\text{SFT}}$) with preference discipline ($\mathcal{L}_{\text{OR}}$).

Step 7: Real-Time Runtime Inference and Swarm Execution (From user prompt to screen: 7 live steps)

When a user submits a prompt on the website during live operation, the system executes this unified 7-step sequence:

1. **Capture User Input:** The user submits objective $\mathcal{T}_{\text{user}}$ via the Next.js web interface, which sends an HTTP POST request to the FastAPI Hive Bridge backend.
2. **Extract Query Features:** Convert the input text into a numerical feature vector combining keyword frequencies (TF-IDF), semantic embeddings, and metadata:

$$\mathbf{x}_{\text{query}} = [\mathbf{v}_{\text{TF-IDF}}(\mathcal{T}_{\text{user}}) \parallel \mathbf{e}_{\text{dense}}(\mathcal{T}_{\text{user}}) \parallel \mathbf{f}_{\text{meta}}]$$

where $\mathbf{f}_{\text{meta}}$ includes query token length, question complexity, and operational flags.

3. **Route Model & Dispatch Agents:** Pass $\mathbf{x}_{\text{query}}$ through the trained classifier to select the best LLM provider and activate the required worker agents:

$$\hat{y}_{\text{intent}}, \hat{y}_{\text{provider}} = \text{argmax}(\hat{\mathbf{y}}_{\text{route}}), \quad \hat{\mathbf{y}}_{\text{agents}} = \mathbb{I}(\hat{\mathbf{y}}_{\text{dispatch}} \geq \tau_{\text{dispatch}})$$

Intuition: The router picks the best model for the task (e.g., Groq for speed, OpenAI for reasoning) and determines which agents to wake up.

4. **Build Execution Plan (Task DAG):** Assemble the active agents into a Directed Acyclic Graph (DAG) and sort their execution order:

$$\pi = \text{TopologicalSort}(\mathcal{G}_{\text{active}})$$

Intuition: Topological sorting ensures prerequisite steps (like researching facts) finish before downstream steps (like writing code) start, with zero circular deadlocks.

5. **Execute ReAct Agent Loop with MCP Tools:** For each scheduled agent $v_i \in \pi$, cycle through Reason $\rightarrow$ Act:

$$a_{i,t} = \text{LLM}_{\hat{y}_{\text{provider}}}(\mathcal{T}_{\text{user}}, h_{i,<t}), \quad o_{i,t} = \text{ExecMCP}(a_{i,t})$$

Intuition: The agent thinks about what to do ($a_{i,t}$), calls external tools (MCP endpoints like web browser or code execution), and observes the output ($o_{i,t}$) before taking the next step.

6. **Verify Safety at Policy Gate:** Check candidate response $\hat{y}_{\text{candidate}}$ against the preference safety threshold:

$$\text{Score}_{\text{safety}} = \log \frac{\text{odds}_{\theta}(\hat{y}_{\text{candidate}} \mid \mathcal{T}_{\text{user}})}{\text{odds}_{\theta}(y_{\text{ref}} \mid \mathcal{T}_{\text{user}})}$$

If score $\geq \tau_{\text{safety}}$, approve the output for delivery; otherwise, reroute to Validator agent for revision.

7. **Stream to UI & Persist Locally:** Stream response tokens live to the user interface via Server-Sent Events (SSE):

$$\text{State}_{\text{UI}}^{(t)} = \text{State}_{\text{UI}}^{(t-1)} \cup \{\Delta_{\text{token}}^{(t)}, \text{Card}_{\text{agent}}\}$$

Upon completion, save conversation history and generated artifacts into local browser IndexedDB (Dexie.js) for instant, private rendering via `useLiveQuery`.

Step 8: Model Evaluation and Calibration (*Verifying system accuracy and reliability*)

1. **Testing on Unseen Data:** Test routing accuracy, agent selection, and alignment on holdout validation ($\mathcal{D}_{\text{val}}$) and testing ($\mathcal{D}_{\text{test}}$) sets to ensure the system generalizes well.

2. Key Evaluation Metrics:

- **Classification Accuracy:** Ratio of correct routing predictions over total queries:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

- **Discriminative Power:** Area Under the ROC Curve (ROC-AUC) and Macro-averaged F1 score measuring how reliably the router distinguishes between different task categories.
- **Confidence Calibration (ECE):** Expected Calibration Error measures whether model confidence matches real accuracy:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|$$

Intuition: If the model is 90% confident on 100 queries, it should be correct on exactly 90 of them. Lower ECE means more honest and well-calibrated confidence.

- **Safety Compliance:** Win rate and policy compliance percentage compared to unaligned baseline models.

3. **Model Export:** Save trained GAT weights, routing heads, and LoRA adapters for production runtime.

Step 9: Swarm Telemetry and Interpretability (*Monitoring and continuous improvement*)

- **Attention Weight Visualizations:** Inspect attention coefficients α_{ij} to see which agents and tools collaborate most heavily for each task type (e.g., Coding agent pairing with GitHub and Filesystem).
- **Operational Telemetry:** Track end-to-end task completion time ($\hat{\ell}$), API token costs ($\hat{c}$), and cache hit rates in real-time to maintain low latency and budget efficiency.
- **Continuous Feedback Loop:** Log runtime execution traces to periodically refresh training data ($\mathcal{D}$) for adaptive retraining.

Algorithm Summary

The PrismSpace algorithm connects intelligent routing, safety alignment, and agent execution into three clean stages:

1. **Smart Swarm Routing:** A Graph Attention Network (GAT) models how agents and tools complement one another, automatically sending each user request to the most cost-effective and capable AI provider.
2. **Single-Stage Preference Alignment:** Odds Ratio Preference Optimization (ORPO) directly teaches models to generate helpful and safe answers without the slow overhead of training separate reward models.
3. **Autonomous Reactive Execution:** Live requests are scheduled as step-by-step plans (DAGs), executed by agents using tools (ReAct + MCP), verified by a safety gate, and streamed in real time to the browser with private local storage (IndexedDB).